

VITyarthi - Build Your Own Project

MathMate

A Menu-Driven Problem Solving Toolkit with Algorithm Efficiency Analysis

Course	CSE1021 - Introduction to Problem Solving and Programming
Student name	Harshvardhan Singh Thakur
Registration number	26BCE10798
Faculty	Adarsh patel
Slot / Section	A11+A12+A13+A14+D11+D12
Submission date	September 30, 2026
GitHub repository	https://github.com/harshvardhansinghthakur135-maker/mathmate-cse1021

Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions and Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings and Key Takeaways
13. Future Enhancements
14. References

1. Introduction

Learning to program in a first-year course means practising many small algorithms: reversing a number, converting between number systems, finding a GCD, factorising an integer, or locating the Kth smallest element of a list. These are normally written as separate, throw-away programs, and the improvement offered by a better algorithm is rarely visible on small inputs.

MathMate is a menu-driven Python program that brings these algorithms into one tool. It has four modules that correspond to units of the CSE1021 syllabus, validates every input, reports errors clearly, logs activity, and includes an Efficiency Lab that runs a simple and an improved algorithm side by side and counts their steps. The program uses only functions, loops, conditionals, lists, tuples, sets and dictionaries, which are exactly the tools taught in the course. It has no dependency on external libraries.

Objectives

- Apply the fundamental algorithms of Units 3, 4 and 5 in one working program.
- Demonstrate top-down design: a menu decomposed into modules, and modules into small functions.
- Show the effect of algorithm choice on efficiency using step counts (Unit 1 analysis of algorithms, Unit 5 time trade-off).
- Make the program robust: validated input, error handling and logging.
- Verify the program with an automated test suite and document it fully.

Scope and Target Users

In scope: a command-line program with four modules, validation, logging, tests and documentation.

Out of scope: graphical or web interface, database, user accounts, and topics outside the syllabus (for example object-oriented programming or recursion). **Target users:** first-year CSE1021 students, teachers demonstrating algorithm efficiency, and beginners who want to check hand-calculated answers.

2. Problem Statement

Beginners practise classic algorithms in isolated programs, with no single place to try them on their own inputs, no friendly handling of wrong input, and no way to see why one method is better than another. The project must provide one validated, easy-to-use tool that (a) implements the core syllabus algorithms, (b) rejects or explains invalid input instead of crashing, and (c) makes the efficiency difference between a simple and an improved algorithm visible.

3. Functional Requirements

ID	Requirement	Module
FR1	Swap values, count and sum digits, factorial, Fibonacci series, reverse and palindrome check, base conversion (2-16), character codes	Number Basics
FR2	Integer square root, smallest divisor / prime check, GCD and LCM, prime sieve, prime factors, LCG pseudo-random numbers, fast power, nth Fibonacci	Factoring
FR3	List statistics, reverse, counting, duplicate removal on an ordered list, partition, Kth smallest, frequency table, set operations	Array Analyzer
FR4	Compare simple and improved algorithms by step count and verify that both give the same answer	Efficiency Lab
FR5	Validate every input (type and range) and ask again when invalid	validation.py
FR6	Show readable error messages for invalid mathematical input	menu.py
FR7	Record every action and error in a log file	logger.py

Input: menu choices, integers, lists of integers, text. **Processing:** the algorithm functions. **Output:** results and messages printed on the terminal; log lines in `logs/mathmate.log`.

4. Non-functional Requirements

ID	Type	Requirement	How it is met
NFR1	Usability	Numbered menus, prompts show valid ranges, wrong input is re-asked	menu.py, validation.py (tests T63-T65)
NFR2	Reliability / error handling	Invalid input or invalid mathematics never crashes the program	try/except ValueError in run_submenu, EOF handling (T66, T70)
NFR3	Performance	Every menu operation finishes almost instantly	Measured once on the build machine: sieve to 100000 = 8.7 ms; $2^{2000} < 1$ ms; factorial(500) = 0.1 ms. Values vary by machine.
NFR4	Maintainability	Small single-purpose functions, docstrings, logic separated from user interface	7 source modules; algorithm modules contain no input() or print()
NFR5	Logging	Actions and errors recorded with timestamps	logger.py
NFR6	Resource efficiency / security	Input sizes are capped so a user cannot freeze the program; no eval()	read_int(..., minimum, maximum) in menu.py
NFR7	Portability	Runs on any OS with Python 3.8+, no packages	requirements.txt

5. System Architecture

MathMate uses a four-layer structure: **presentation** (main.py, menu.py), **validation** (validation.py), **algorithm modules** (basic_algorithms.py, factoring.py, array_tools.py, efficiency.py) and **support** (logger.py). Algorithm modules never read from the keyboard or print, so they can be tested directly. The menu is the only component that talks to the user and the only one that imports all modules; this was checked against the import statements in the source.

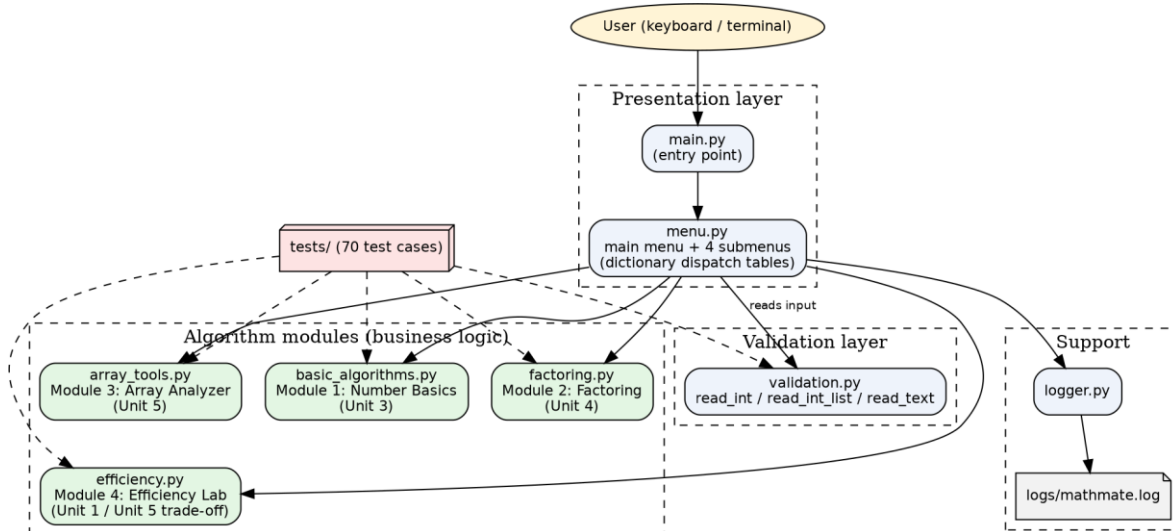


Figure 1: System architecture diagram

6. Design Diagrams

6.1 Use Case Diagram

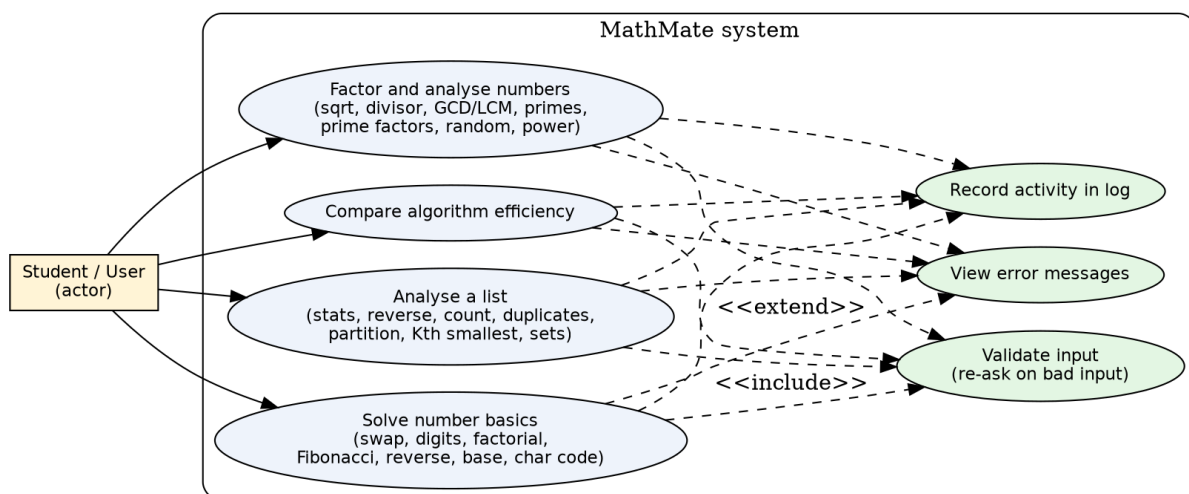


Figure 2: Use case diagram

6.2 Workflow Diagram

The workflow shows the loop of the program: main menu, submenu, input validation, algorithm execution, and the two outcomes (result or error message), both of which return to the submenu.

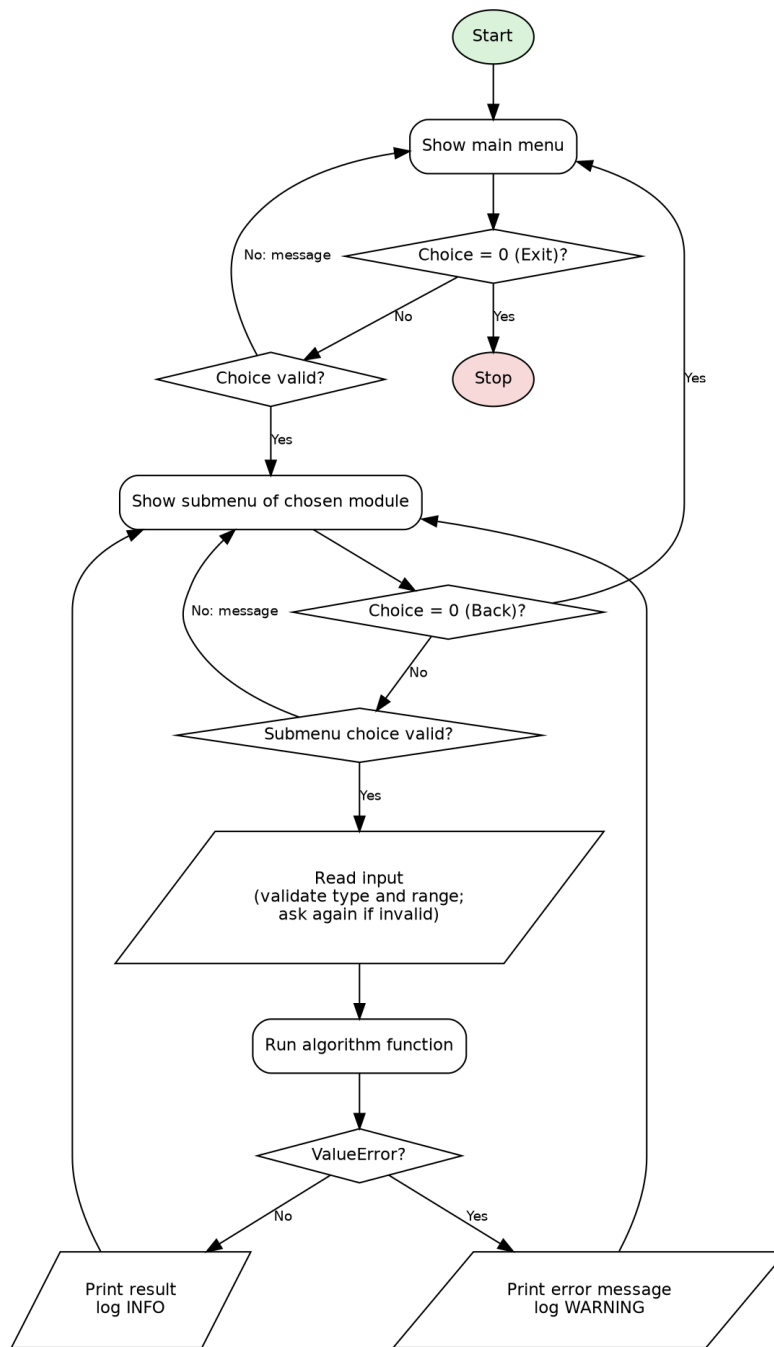


Figure 3: Workflow diagram

6.3 Sequence Diagram

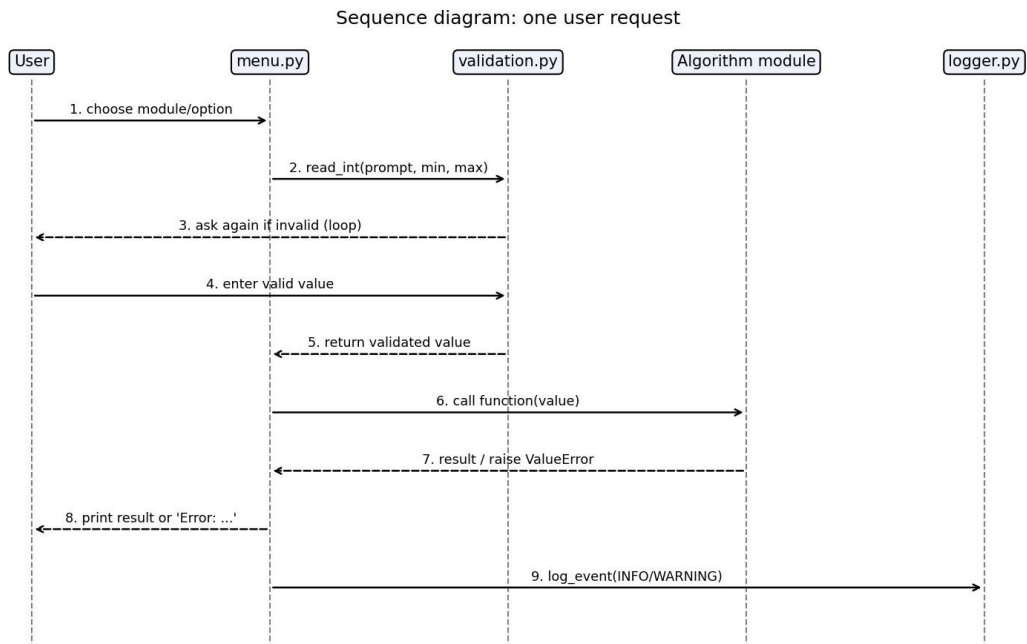


Figure 4: Sequence diagram for a single user request

6.4 Component Diagram

Because the syllabus does not include object-oriented programming, the program has no classes. A component (module) diagram is used instead of a class diagram, as permitted by the project guidelines.

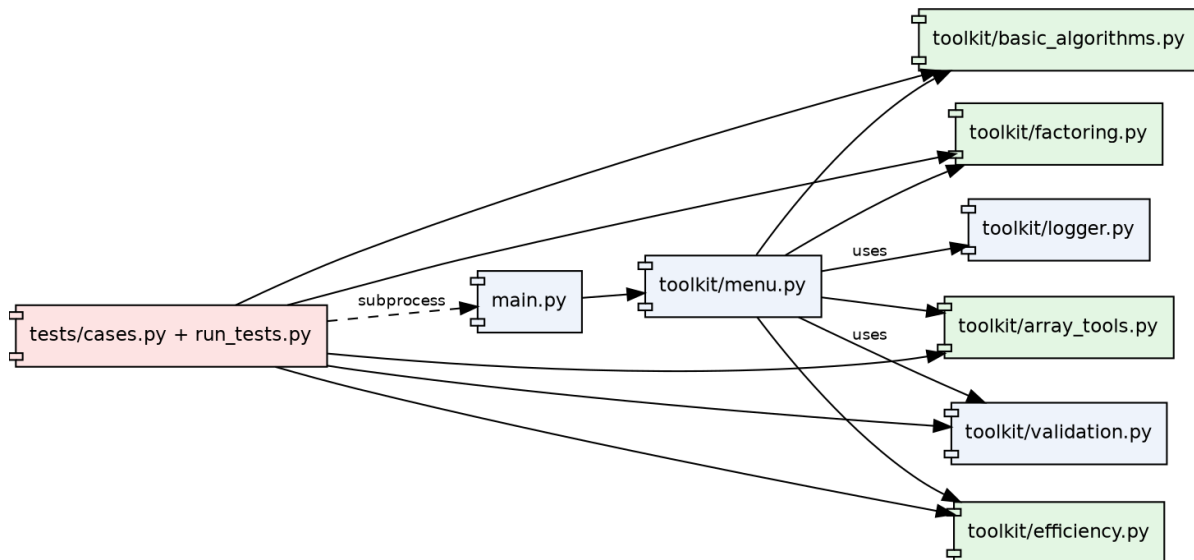


Figure 5: Component diagram

6.5 ER Diagram / Database Schema

Not applicable. MathMate has no database. Its only storage is the plain-text log file `logs/mathmate.log` with one line per event in the form `timestamp | LEVEL | message`.

7. Design Decisions and Rationale

Decision	Chosen	Alternatives	Reason
Interface	Text menus	tkinter GUI, web app	GUI/web are outside the syllabus; CLI keeps focus on algorithms
Structure	Functions in modules	Classes (OOP)	OOP is not in the syllabus; guidelines accept a component diagram
Menu control	Dictionary {key: (label, function)}	long if/elif chain	Uses dictionaries (Unit 5); adding an option is one line
Error strategy	Algorithms raise ValueError; menu catches it	Return error codes	Keeps algorithms clean; one place prints messages
Input safety	Central validation.py re-asks until valid	Validate inside each function	Avoids duplicate code and crashes
Efficiency measure	Count steps	Time with the time module	Step counts are identical on every computer and match Unit 1 analysis
Tests	Data table + runner (plain Python)	unittest / pytest	Not in syllabus, nothing to install, output matches the required test-case format
Storage	Text log only	JSON / SQLite	No data needs to be saved, so no ER diagram is required

Course Concept Mapping

Course concept (unit)	Feature	Where
Top-down design (1)	Menu -> modules -> functions	main.py, menu.py
Flowcharts / pseudo-code (1)	Workflow flowchart	Figure 3
Program verification (1)	Both methods must agree; property checks over ranges	menu.py do_lab_*; tests T51, T53, T60, T61
Efficiency and analysis of algorithms (1)	Step counters	efficiency.py
Tuple assignment, expressions (2)	swap, Euclid GCD, Fibonacci update	basic_algorithms.py, factoring.py
Functions, parameters, modules (2)	All features	all files
Exchange, counting, summation, factorial, Fibonacci, reverse, base and character conversion (3)	Module 1	basic_algorithms.py
Conditionals, loops, continue, pass (3)	Menus, validation loops	menu.py, validation.py, logger.py
Square root, divisors, GCD, primes, prime factors, random numbers, large powers (4)	Module 2	factoring.py
Array reversal, counting, maximum, duplicates in ordered array, partition, Kth smallest (5)	Module 3	array_tools.py
Lists, tuples, sets, dictionaries (5)	partition (tuple), set_operations (sets), frequency_table and menus (dictionaries)	array_tools.py, menu.py
Time trade-off (5)	Nested loops vs set for duplicates	efficiency.py
Scratch / Raptor (6)	Not used in the program (see Future Enhancements)	-

8. Implementation Details

File	Purpose	Lines
main.py	Entry point, starts the main menu	5
toolkit/menu.py	All menus and prompts; dictionary dispatch; error catching	302
toolkit/validation.py	read_int, read_int_list, read_text, parse_int_list	57
toolkit/logger.py	Timestamped log file writer	20
toolkit/basic_algorithms.py	Module 1: 11 functions (Unit 3)	124
toolkit/factoring.py	Module 2: 11 functions (Unit 4)	129
toolkit/array_tools.py	Module 3: 10 functions (Unit 5)	122
toolkit/efficiency.py	Module 4: 7 functions with step counters	100
tests/cases.py, tests/run_tests.py	70 test cases and the runner	143 + 63

Key algorithms

Euclid's GCD (Unit 4) uses tuple assignment (Unit 2). Hand trace of `gcd(48, 18)`: `(48,18) -> (18,12) -> (12,6) -> (6,0)`, answer 6.

```
def gcd(a, b):
    if a < 0 or b < 0:
        raise ValueError("GCD needs non-negative numbers.")
    while b != 0:
        a, b = b, a % b
    return a
```

Fast power squares the base and halves the exponent, so 2^{1000} needs 16 multiplications instead of 1000 (verified in the Efficiency Lab, Section 9).

```
def fast_power(base, exponent):
    result = 1
    while exponent > 0:
        if exponent % 2 == 1:
            result *= base
        base *= base
        exponent //= 2
    return result
```

Smallest divisor tests 2 and then only odd numbers up to the square root, because any composite number has a divisor not larger than its square root. **Prime factors** repeatedly divide by the smallest divisor. The **sieve of Eratosthenes** crosses out multiples of each prime up to the square root of the limit. **Integer square root** uses Newton's iteration with integer division. The **pseudo-random generator** is a Linear Congruential Generator ($a = 1103515245$, $c = 12345$, $m = 2^{31}$), so the same seed always gives the same sequence. It is not suitable for security use.

Validation and error handling. `read_int` repeats until the text is an integer inside the allowed range. Algorithm functions raise `ValueError` for invalid mathematics; `run_submenu` catches it, prints the message, logs a warning and returns to the menu. Closed input (EOF) ends the program cleanly.

```
try:
    handler()
    log_event("INFO", f"{title} > {label}")
except ValueError as error:
    print(f"  Error: {error}")
    log_event("WARNING", f"{title} > {label}: {error}")
```

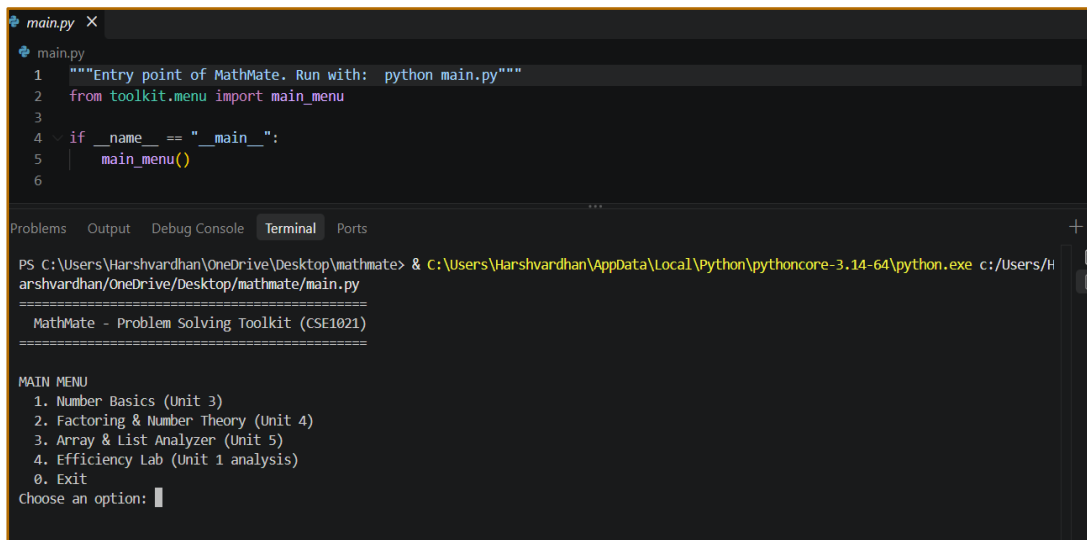
Data structures. Lists hold inputs and results; tuples are returned as pairs such as (divisor, steps); sets implement union/intersection and fast duplicate detection; dictionaries store frequency tables, prime exponents and the menus.

9. Screenshots / Results

The console output below was produced by running the real program code with the shown inputs (generated by scripts/make_sample_runs.py; menu listings removed for brevity). Screenshot boxes must be replaced with the student's own screenshots taken while running the program; the checklist is in docs/screenshot_checklist.md.

9.1 Number system conversion and error handling (Module 1)

```
MathMate - Problem Solving Toolkit (CSE1021)
Choose an option: 1
Choose an option: 6
    Decimal number: 255
    Target base (2-16): 16
    255 in base 16 = FF
Choose an option: 7
    Source base (2-16): 2
    Number in that base: 1092
    Error: '9' is not a valid digit in base 2.
Choose an option: 0
Choose an option: 0
Goodbye!
```



The screenshot shows a Python IDE with a file named `main.py` open. The code in the editor is as follows:

```
1 """Entry point of MathMate. Run with: python main.py"""
2 from toolkit.menu import main_menu
3
4 if __name__ == "__main__":
5     main_menu()
6
```

The terminal window at the bottom shows the command prompt output:

```
PS C:\Users\Harshvardhan\OneDrive\Desktop\mathmate> & C:\Users\Harshvardhan\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/H
arshvardhan/OneDrive/Desktop/mathmate/main.py

MathMate - Problem Solving Toolkit (CSE1021)

MAIN MENU
1. Number Basics (Unit 3)
2. Factoring & Number Theory (Unit 4)
3. Array & List Analyzer (Unit 5)
4. Efficiency Lab (Unit 1 analysis)
0. Exit
Choose an option: 
```

```
main.py X
main.py
1 """Entry point of MathMate. Run with: python main.py"""
2 from toolkit.menu import main_menu
3
4 if __name__ == "__main__":
5     main_menu()
6

Problems Output Debug Console Terminal Ports

2. Factoring & Number Theory (Unit 4)
3. Array & List Analyzer (Unit 5)
4. Efficiency Lab (Unit 1 analysis)
0. Exit
Choose an option: 1

--- Number Basics (Unit 3) ---
1. Swap two values
2. Count digits and sum of digits
3. Factorial
4. Fibonacci series
5. Reverse a number / palindrome check
6. Decimal to another base
7. Another base to decimal
8. Character to number codes
0. Back
Choose an option: 6
Decimal number: 255
Target base (2-16): 16
255 in base 16 = FF
```

9.2 Factoring and large power (Module 2)

```
MathMate - Problem Solving Toolkit (CSE1021)
Choose an option: 2
Choose an option: 5
Enter a number (>= 2): 360
Prime factors: [2, 2, 2, 3, 3, 5]    ->    360 = 2^3 x 3^2 x 5
Choose an option: 7
Base: 2
Exponent (0-2000): 100
2^100 = 1267650600228229401496703205376   (31 digits)
Choose an option: 0
Choose an option: 0
Goodbye!
```

```
main.py X
main.py
1  """Entry point of MathMate. Run with: python main.py"""
2  from toolkit.menu import main_menu
3
4  if __name__ == "__main__":
5      main_menu()
6

Problems Output Debug Console Terminal Ports
1. Number Basics (Unit 3)
2. Factoring & Number Theory (Unit 4)
3. Array & List Analyzer (Unit 5)
4. Efficiency Lab (Unit 1 analysis)
0. Exit
Choose an option: 2

--- Factoring & Number Theory (Unit 4) ---
1. Integer square root
2. Smallest divisor / prime check
3. GCD and LCM
4. Generate primes (sieve)
5. Prime factors
6. Pseudo-random numbers
7. Raise a number to a large power
8. nth Fibonacci number
0. Back
Choose an option: 5
Enter a number (>= 2): 360
Prime factors: [2, 2, 2, 3, 3, 5] -> 360 = 2^3 x 3^2 x 5
```

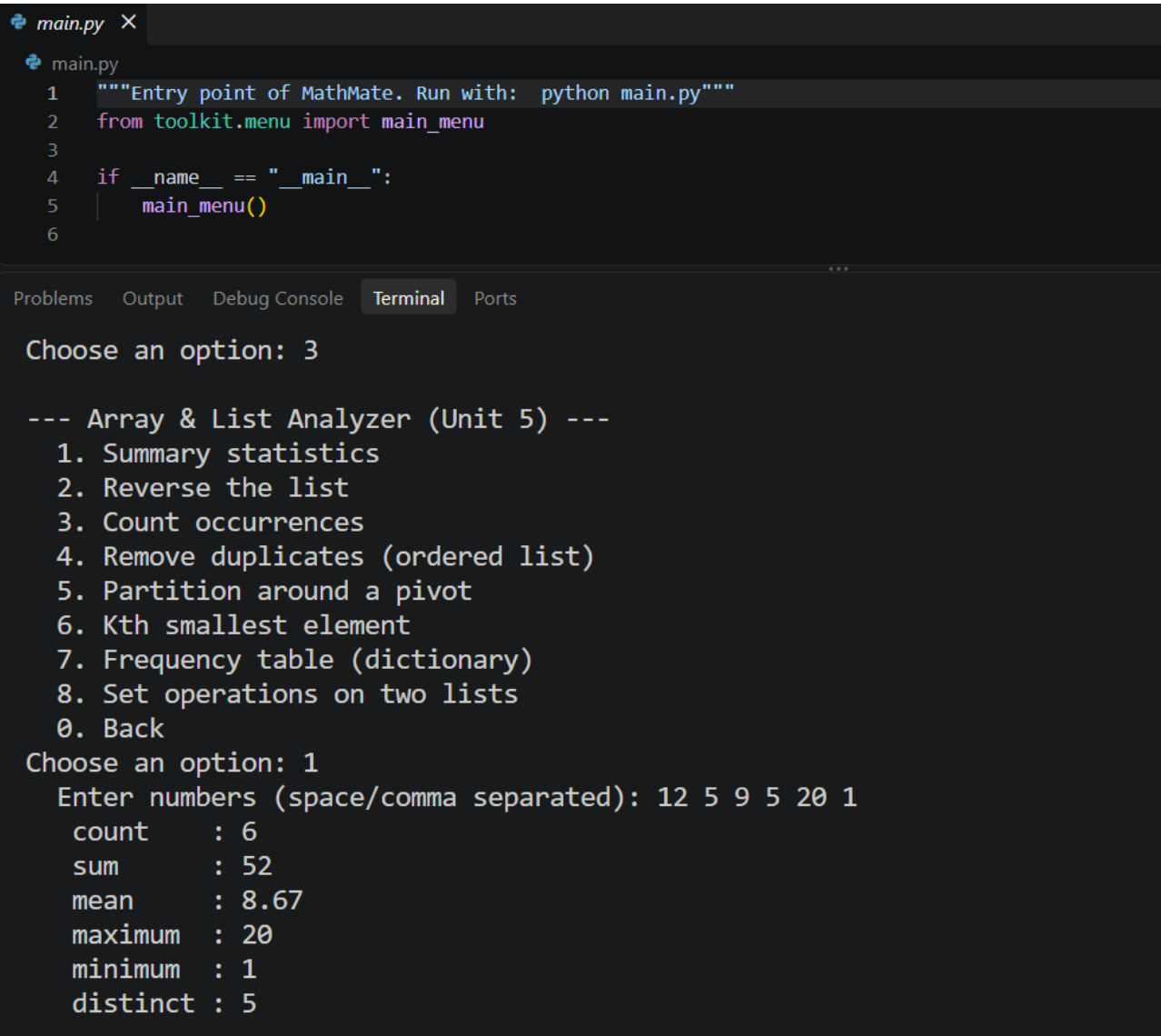
```
main.py X
main.py
1 """Entry point of MathMate. Run with: python main.py"""
2 from toolkit.menu import main_menu
3
4 if __name__ == "__main__":
5     main_menu()
6

Problems Output Debug Console Terminal Ports
2. Factoring & Number Theory (Unit 4)
3. Array & List Analyzer (Unit 5)
4. Efficiency Lab (Unit 1 analysis)
0. Exit
Choose an option: 2

--- Factoring & Number Theory (Unit 4) ---
1. Integer square root
2. Smallest divisor / prime check
3. GCD and LCM
4. Generate primes (sieve)
5. Prime factors
6. Pseudo-random numbers
7. Raise a number to a large power
8. nth Fibonacci number
0. Back
Choose an option: 7
Base: 2
Exponent (0-2000): 100
2^100 = 1267650600228229401496703205376 (31 digits)
```

9.3 Array analysis (Module 3)

```
MathMate - Problem Solving Toolkit (CSE1021)
Choose an option: 3
Choose an option: 1
Enter numbers (space/comma separated): 12 5 9 5 20 1
count      : 6
sum        : 52
mean       : 8.67
maximum    : 20
minimum    : 1
distinct   : 5
Choose an option: 6
Enter numbers: 9 4 7 1 8
k (1-5): 3
Kth smallest element (k = 3) is 7
Choose an option: 0
Choose an option: 0
Goodbye!
```



The screenshot shows a code editor with a file named `main.py`. The code in the editor is as follows:

```
1 """Entry point of MathMate. Run with: python main.py"""
2 from toolkit.menu import main_menu
3
4 if __name__ == "__main__":
5     main_menu()
6
```

Below the code editor, the terminal output is displayed. It shows the program running and the user interacting with the menu:

```
Choose an option: 3

--- Array & List Analyzer (Unit 5) ---
1. Summary statistics
2. Reverse the list
3. Count occurrences
4. Remove duplicates (ordered list)
5. Partition around a pivot
6. Kth smallest element
7. Frequency table (dictionary)
8. Set operations on two lists
0. Back
Choose an option: 1
Enter numbers (space/comma separated): 12 5 9 5 20 1
count      : 6
sum        : 52
mean       : 8.67
maximum    : 20
minimum    : 1
distinct   : 5
```

```
main.py X
main.py
1  """Entry point of MathMate. Run with: python main.py"""
2  from toolkit.menu import main_menu
3
4  if __name__ == "__main__":
5      main_menu()
6
Problems  Output  Debug Console  Terminal  Ports

--- Array & List Analyzer (Unit 5) ---
1. Summary statistics
2. Reverse the list
3. Count occurrences
4. Remove duplicates (ordered list)
5. Partition around a pivot
6. Kth smallest element
7. Frequency table (dictionary)
8. Set operations on two lists
0. Back
Choose an option: 6
Enter numbers: 9 4 7 1 8
k (1-5): 3
Kth smallest element (k = 3) is 7

--- Array & List Analyzer (Unit 5) ---
1. Summary statistics
2. Reverse the list
3. Count occurrences
4. Remove duplicates (ordered list)
```

9.4 Efficiency Lab (Module 4)

MathMate - Problem Solving Toolkit (CSE1021)

Choose an option: 4

Choose an option: 1

Number to test (≥ 2 , try 999983): 999983

Smallest divisor of 999983

Method	Steps
Try every number	999982
Odd numbers up to $\sqrt{n}$	500
Improved method used 2000.0x fewer steps.	
Verification: both methods agree	

Choose an option: 2

Base: 2

Exponent (0-5000): 1000

2^{1000}

Method	Steps
Repeated multiplication	1000
Repeated squaring	16
Improved method used 62.5x fewer steps.	
Verification: both methods agree	

Choose an option: 3

Enter numbers: 5 3 9 3

Duplicate check

Method	Steps
Nested loops (comparisons)	5
Set (lookups, extra memory)	4
Improved method used 1.2x fewer steps.	
Duplicates present: Yes	

Choose an option: 0

Choose an option: 0

Goodbye!

Result analysis. For $n = 999983$ (a prime) the simple method needed 999982 steps and the improved method 500 steps. For 2^{1000} , repeated multiplication needed 1000 multiplications and repeated squaring 16. For the list $[5, 3, 9, 3]$ the set method used 4 lookups against 5 comparisons; the difference is small for short lists but grows quickly because nested loops need $O(n^2)$ comparisons in the worst case (verified for a 100-element list without duplicates: 4950 comparisons vs 100 lookups). The price of the set method is extra memory (time trade-off).

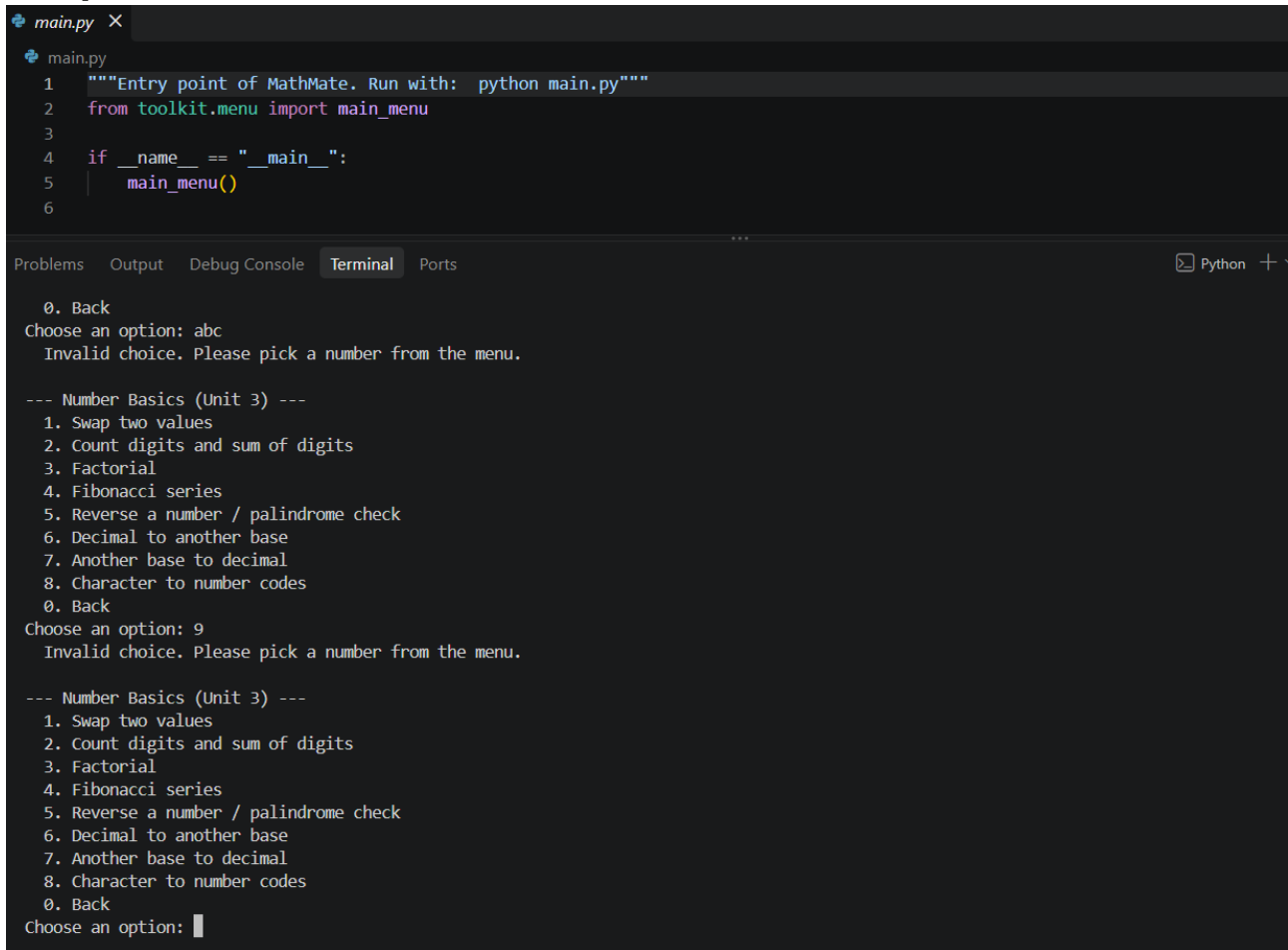
```
MAIN MENU
1. Number Basics (Unit 3)
2. Factoring & Number Theory (Unit 4)
3. Array & List Analyzer (Unit 5)
4. Efficiency Lab (Unit 1 analysis)
0. Exit
Choose an option: 4

--- Efficiency Lab (Unit 1 analysis) ---
1. Smallest divisor: naive vs sqrt method
2. Power: multiplication vs squaring
3. Duplicates: nested loops vs set (time trade-off)
0. Back
Choose an option: 1
Number to test (>= 2, try 999983): 999983
Smallest divisor of 999983
Method                               Steps
Try every number                      999982
Odd numbers up to sqrt(n)             500
Improved method used 2000.0x fewer steps.
Verification: both methods agree

--- Efficiency Lab (Unit 1 analysis) ---
1. Smallest divisor: naive vs sqrt method
2. Power: multiplication vs squaring
3. Duplicates: nested loops vs set (time trade-off)
0. Back
```

9.5 Input validation

```
MathMate - Problem Solving Toolkit (CSE1021)
Choose an option: 1
Choose an option: 3
Enter n (0-500): abc
Invalid input: please enter a whole number.
Enter n (0-500): 999
Value must be at most 500.
Enter n (0-500): 5
5! = 120
Choose an option: 0
Choose an option: 0
Goodbye!
```



The screenshot shows a Python IDE with a file named `main.py` open. The code in the editor is as follows:

```
1 """Entry point of MathMate. Run with: python main.py"""
2 from toolkit.menu import main_menu
3
4 if __name__ == "__main__":
5     main_menu()
6
```

The terminal window at the bottom shows the execution of the program. It displays a menu with options 0 through 8, where 0 is 'Back' and 1-8 are various mathematical operations. The user enters 'abc' for the first option, which results in an 'Invalid choice' message. Then, the user enters '9', which also results in an 'Invalid choice' message. The terminal output is as follows:

```
0. Back
Choose an option: abc
Invalid choice. Please pick a number from the menu.

--- Number Basics (Unit 3) ---
1. Swap two values
2. Count digits and sum of digits
3. Factorial
4. Fibonacci series
5. Reverse a number / palindrome check
6. Decimal to another base
7. Another base to decimal
8. Character to number codes
0. Back
Choose an option: 9
Invalid choice. Please pick a number from the menu.

--- Number Basics (Unit 3) ---
1. Swap two values
2. Count digits and sum of digits
3. Factorial
4. Fibonacci series
5. Reverse a number / palindrome check
6. Decimal to another base
7. Another base to decimal
8. Character to number codes
0. Back
Choose an option: 
```

```

mathmate.log
File Edit View

2026-09-30 17:03:07 | INFO | Program started
2026-09-30 17:03:49 | INFO | Number Basics (Unit 3) > Factorial
2026-09-30 17:04:00 | INFO | Program ended
2026-09-30 17:04:13 | INFO | Program started
2026-09-30 17:04:30 | INFO | Factoring & Number Theory (Unit 4) > Prime factors
2026-09-30 17:04:57 | INFO | Array & List Analyzer (Unit 5) > Remove duplicates (ordered list)
2026-09-30 17:07:08 | INFO | Program ended
2026-09-30 17:07:52 | INFO | Program started
2026-09-30 17:07:52 | INFO | Number Basics (Unit 3) > Factorial
2026-09-30 17:07:52 | INFO | Program ended
2026-09-30 17:07:53 | INFO | Program started
2026-09-30 17:07:53 | INFO | Program ended
2026-09-30 17:07:53 | INFO | Program started
2026-09-30 17:07:53 | INFO | Number Basics (Unit 3) > Factorial
2026-09-30 17:07:53 | INFO | Program ended
2026-09-30 17:07:53 | INFO | Program started
2026-09-30 17:07:53 | INFO | Number Basics (Unit 3) > Factorial
2026-09-30 17:07:53 | INFO | Program ended
2026-09-30 17:07:53 | INFO | Program started
2026-09-30 17:07:53 | WARNING | Number Basics (Unit 3) > Another base to decimal: '9' is not a valid digit in base 2.
2026-09-30 17:07:53 | INFO | Program ended
2026-09-30 17:07:53 | INFO | Program started
2026-09-30 17:07:53 | INFO | Factoring & Number Theory (Unit 4) > Prime factors
2026-09-30 17:07:53 | INFO | Program ended
2026-09-30 17:07:53 | INFO | Program started
2026-09-30 17:07:53 | INFO | Array & List Analyzer (Unit 5) > Kth smallest element
2026-09-30 17:07:53 | INFO | Program ended
2026-09-30 17:07:53 | INFO | Program started
2026-09-30 17:07:53 | INFO | Efficiency Lab (Unit 1 analysis) > Smallest divisor: naive vs sqrt method
2026-09-30 17:07:53 | INFO | Program ended
2026-09-30 17:07:53 | INFO | Program started
2026-09-30 17:07:53 | INFO | Program ended (input closed)
2026-09-30 17:07:54 | INFO | Program started
2026-09-30 17:07:54 | INFO | Number Basics (Unit 3) > Factorial
2026-09-30 17:07:54 | INFO | Program ended
2026-09-30 17:07:54 | INFO | Program started
2026-09-30 17:07:54 | INFO | Program ended
2026-09-30 17:07:54 | INFO | Program started
2026-09-30 17:07:54 | INFO | Program ended
2026-09-30 17:07:54 | INFO | Number Basics (Unit 3) > Factorial
2026-09-30 17:07:54 | INFO | Program ended

Ln 1, Col 1 8,372 characters Plain text 100% Windows (CRLF) UTF-8

```

```

run_tests.py
tests > run_tests.py > ...
1 """Run every test case and print a results table.
2
3 Usage (from the project folder): python tests/run_tests.py
4 Also writes docs/test_results.md with the real results of this run.
5 """
6 import os
7 import sys
...

Problems Output Debug Console Terminal Ports
Python + - - - - -

| T49 | Set operations | ([1, 2, 3], [2, 3, 4]) | {'union': [1, 2, 3, 4], 'intersecti... | {'union': [1, 2, 3, 4], 'intersecti... | PASS |
| T50 | Summary of [2,4,6] | ([2, 4, 6],) | {'count': 3, 'sum': 12, 'mean': 4.0... | {'count': 3, 'sum': 12, 'mean': 4.0... | PASS |
| T51 | Naive vs improved divisor agree (2..500) | (none) | True | True | PASS |
| T52 | Improved divisor uses fewer steps for 999983 | (none) | True | True | PASS |
| T53 | Both power methods agree | (none) | True | True | PASS |
| T54 | Fast power steps for exponent 1000 | (2, 1000) | (1071508607186267320948425049060001... | (1071508607186267320948425049060001... | PASS |
| T55 | Nested duplicate check, no duplicates | ([1, 2, 3, 4],) | (False, 6) | (False, 6) | PASS |
| T56 | Set duplicate check finds duplicate | ([1, 2, 1, 4],) | (True, 3) | (True, 3) | PASS |
| T57 | Parse '4, 7 2' | ('4, 7 2',) | [4, 7, 2] | [4, 7, 2] | PASS |
| T58 | Parse empty text | (' ',) | raises ValueError | ValueError | PASS |
| T59 | Parse text with a letter | ('1 x 3',) | raises ValueError | ValueError | PASS |
| T60 | integer_sqrt property for 0..2000 | (none) | True | True | PASS |
| T61 | Product of prime factors equals n (2..1000) | (none) | True | True | PASS |
| T62 | Menu: factorial of 5 through the UI | keys: '1\n3\n5\n0\n0\n' | contains '5! = 120' | contains text | PASS |
| T63 | Menu: invalid main choice is rejected | keys: '9\n0\n' | contains 'Invalid choice' | contains text | PASS |
| T64 | Menu: letters instead of number are re-asked | keys: '1\n3\nabc\n4\n0\n0\n' | contains 'please enter a whole number' | contains text | PASS |
| T65 | Menu: out-of-range value is re-asked | keys: '1\n3\n999\n3\n0\n0\n' | contains 'at most 500' | contains text | PASS |
| T66 | Menu: bad digit shows error, app continues | keys: '1\n7\n2\n1092\n0\n0\n' | contains 'Error:' | contains text | PASS |
| T67 | Menu: prime factors of 360 through the UI | keys: '2\n5\n360\n0\n0\n' | contains '2^3 x 3^2 x 5' | contains text | PASS |
| T68 | Menu: kth smallest through the UI | keys: '3\n6\n9 4 7 1 8\n3\n0\n0\n' | contains '(k = 3) is 7' | contains text | PASS |
| T69 | Menu: efficiency lab shows verification | keys: '4\n1\n97\n0\n0\n' | contains 'both methods agree' | contains text | PASS |
| T70 | Menu: closed input ends program cleanly | keys: '1\n3\n' | contains 'Input closed' | contains text | PASS |

Total: 70 Passed: 70 Failed: 0
PS C:\Users\Harshvardhan\OneDrive\Desktop\mathmate>

```

10. Testing Approach

Testing is automated with a plain-Python runner (tests/run_tests.py). Each test case is one row of data: ID, description, function, input and expected result (a value, an expected exception, or text that must appear in the program output). Test types: **unit tests** of every algorithm function, **edge cases** (0, empty list, k out of range, base limits), **error tests** (expected ValueError), **verification tests** that compare two independent methods or check a mathematical property over a whole range (integer square root for 0-2000, product of prime factors for 2-1000, naive vs improved methods), and **integration tests** that start main.py and type keystrokes into the menu.

Result of the final run: 70 test cases executed, 70 passed, 0 failed.

Bugs found by testing. (1) In the first run, T54 failed: the expected step count I wrote (12) was wrong; for exponent 1000 (binary 1111101000) fast power needs 10 squarings and 6 multiplications = 16. The test data was corrected; the code was already right. (2) While reviewing menu output, the message for the Kth smallest element read "3th"; it was rewritten as "Kth smallest element (k = 3) is ...". (3) While drawing the architecture diagram, an arrow suggested that efficiency.py depends on factoring.py; the imports showed this was false and the arrow was removed.

Test cases

ID	Description	Input	Expected	Actual	Status
Test ID	Description	Input	Expected Result	Actual Result	Status
T01	Swap two values	(3, 9)	(9, 3)	(9, 3)	PASS
T02	Count digits of 12345	(12345,)	5	5	PASS
T03	Count digits of 0 (edge case)	(0,)	1	1	PASS
T04	Sum of digits of 9875	(9875,)	29	29	PASS
T05	Factorial of 5	(5,)	120	120	PASS
T06	Factorial of 0 (edge case)	(0,)	1	1	PASS
T07	Factorial of negative number	(-3,)	raises ValueError	ValueError	PASS
T08	First 8 Fibonacci terms	(8,)	[0, 1, 1, 2, 3, 5, 8, 13]	[0, 1, 1, 2, 3, 5, 8, 13]	PASS
T09	Reverse 1230 (trailing zero dropped)	(1230,)	321	321	PASS
T10	Palindrome check 12321	(12321,)	True	True	PASS
T11	Palindrome check 123	(123,)	False	False	PASS
T12	255 to binary	(255, 2)	'11111111'	'11111111'	PASS
T13	255 to hexadecimal	(255, 16)	'FF'	'FF'	PASS
T14	Binary 1010 to decimal	('1010', 2)	10	10	PASS
T15	Digit '9' invalid in base 2	('1092', 2)	raises ValueError	ValueError	PASS
T16	Base 20 rejected	(10, 20)	raises ValueError	ValueError	PASS
T17	Character codes of 'Hi'	('Hi',)	[('H', 72), ('i', 105)]	[('H', 72), ('i', 105)]	PASS
T18	Integer sqrt of 99	(99,)	9	9	PASS
T19	Integer sqrt of 0 (edge case)	(0,)	0	0	PASS
T20	Integer sqrt of negative	(-4,)	raises ValueError	ValueError	PASS
T21	Smallest divisor of 91	(91,)	7	7	PASS
T22	Smallest divisor of prime 97	(97,)	97	97	PASS
T23	GCD of 48 and 18	(48, 18)	6	6	PASS
T24	GCD with zero	(0, 5)	5	5	PASS
T25	LCM of 4 and 6	(4, 6)	12	12	PASS
T26	Primes up to 30	(30,)	[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]	[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]	PASS
T27	Primes up to 1 (edge case)	(1,)	[]	[]	PASS
T28	Prime factors of 360	(360,)	[2, 2, 2, 3, 3, 5]	[2, 2, 2, 3, 3, 5]	PASS
T29	Prime factors of 1 rejected	(1,)	raises ValueError	ValueError	PASS
T30	Factor dictionary of 360	(360,)	{2: 3, 3: 2, 5: 1}	{2: 3, 3: 2, 5: 1}	PASS
T31	LCG gives 5 numbers inside 1..6	(none)	True	True	PASS
T32	LCG is repeatable for same seed	(none)	True	True	PASS
T33	LCG low > high rejected	(1, 3, 9, 2)	raises ValueError	ValueError	PASS
T34	2 to the power 100	(2, 100)	1267650600228229401496703205376	1267650600228229401496703205376	PASS
T35	Power with negative exponent	(2, -1)	raises ValueError	ValueError	PASS
T36	10th Fibonacci number	(10,)	55	55	PASS
T37	Reverse [1,2,3,4]	([1, 2, 3, 4],)	[4, 3, 2, 1]	[4, 3, 2, 1]	PASS
T38	Reverse empty list (edge case)	([],)	[]	[]	PASS
T39	Count occurrences of 2	([2, 5, 2, 2], 2)	3	3	PASS
T40	Maximum of [4,-1,9,3]	([4, -1, 9, 3],)	9	9	PASS
T41	Maximum of empty list	([],)	raises ValueError	ValueError	PASS
T42	Remove duplicates from sorted list	([1, 1, 2, 3, 3, 3],)	[1, 2, 3]	[1, 2, 3]	PASS
T43	Unsorted list rejected	([3, 1, 2],)	raises ValueError	ValueError	PASS

ID	Description	Input	Expected	Actual	Status
T44	Partition around 5	([7, 2, 5, 9, 5, 1], 5)	([2, 1], [5, 5], [7, 9])	([2, 1], [5, 5], [7, 9])	PASS
T45	3rd smallest of [9,4,7,1,8]	([9, 4, 7, 1, 8], 3)	7	7	PASS
T46	k larger than list size	([1, 2], 5)	raises ValueError	ValueError	PASS
T47	k = 0 rejected	([1, 2], 0)	raises ValueError	ValueError	PASS
T48	Frequency table	([1, 2, 1, 3, 1],)	{1: 3, 2: 1, 3: 1}	{1: 3, 2: 1, 3: 1}	PASS
T49	Set operations	([1, 2, 3], [2, 3, 4])	{'union': [1, 2, 3, 4], 'intersecti...	{'union': [1, 2, 3, 4], 'intersecti...	PASS
T50	Summary of [2,4,6]	([2, 4, 6],)	{'count': 3, 'sum': 12, 'mean': 4.0...	{'count': 3, 'sum': 12, 'mean': 4.0...	PASS
T51	Naive vs improved divisor agree (2..500)	(none)	True	True	PASS
T52	Improved divisor uses fewer steps for 999983	(none)	True	True	PASS
T53	Both power methods agree	(none)	True	True	PASS
T54	Fast power steps for exponent 1000	(2, 1000)	(107150860718626732094 8425049060001...	(107150860718626732094 8425049060001...	PASS
T55	Nested duplicate check, no duplicates	([1, 2, 3, 4],)	(False, 6)	(False, 6)	PASS
T56	Set duplicate check finds duplicate	([1, 2, 1, 4],)	(True, 3)	(True, 3)	PASS
T57	Parse '4, 7 2'	('4, 7 2',)	[4, 7, 2]	[4, 7, 2]	PASS
T58	Parse empty text	(' ',)	raises ValueError	ValueError	PASS
T59	Parse text with a letter	('1 x 3',)	raises ValueError	ValueError	PASS
T60	integer_sqrt property for 0..2000	(none)	True	True	PASS
T61	Product of prime factors equals n (2..1000)	(none)	True	True	PASS
T62	Menu: factorial of 5 through the UI	keys: '1\n3\n5\n0\n0\n'	contains '5! = 120'	contains text	PASS
T63	Menu: invalid main choice is rejected	keys: '9\n0\n'	contains 'Invalid choice'	contains text	PASS
T64	Menu: letters instead of number are re-asked	keys: '1\n3\nabc\n4\n0\n0\n'	contains 'please enter a whole number'	contains text	PASS
T65	Menu: out-of-range value is re-asked	keys: '1\n3\n999\n3\n0\n0\n'	contains 'at most 500'	contains text	PASS
T66	Menu: bad digit shows error, app continues	keys: '1\n7\n2\n1092\n0\n0\n'	contains 'Error:'	contains text	PASS
T67	Menu: prime factors of 360 through the UI	keys: '2\n5\n360\n0\n0\n'	contains '2^3 x 3^2 x 5'	contains text	PASS
T68	Menu: kth smallest through the UI	keys: '3\n6\n9 4 7 1 8\n3\n0\n0\n'	contains '(k = 3) is 7'	contains text	PASS
T69	Menu: efficiency lab shows verification	keys: '4\n1\n97\n0\n0\n'	contains 'both methods agree'	contains text	PASS
T70	Menu: closed input ends program cleanly	keys: '1\n3\n'	contains 'Input closed'	contains text	PASS

11. Challenges Faced

Challenge	Solution
Keeping the project inside the syllabus (no OOP, GUI, database or external libraries)	Function-based modules, CLI menus, standard library only; component diagram instead of class diagram; ER diagram marked not applicable
Showing efficiency without depending on the speed of the computer	Counted steps instead of timing
Keeping logic testable	Algorithm modules contain no input/print; menu layer handles all interaction
Preventing crashes from bad input	Central validation, capped ranges, ValueError handling, EOF handling
Incorrect expected value and wording found during testing	Corrected test T54 and the Kth smallest message (Section 10)
Rendering diagrams	Mermaid could not be rendered in the build environment, so Graphviz and matplotlib were used for the PDF images; Mermaid source is kept in docs/diagrams.md but was not render-tested

12. Learnings and Key Takeaways

- Separating input/output from logic makes programs far easier to test and explain.
- Step counting makes Big-O ideas concrete: the same answer can cost 999982 or 500 steps.
- Tests can be wrong too; a failing test needs investigation before changing the code.
- Validation and clear error messages are part of solving a problem, not extras.
- Choosing simple, explainable solutions (for example Kth smallest by repeated minimum) keeps the project defensible.

Limitations

- Integers only; command-line only; nothing is saved between runs.
- Kth smallest runs in $O(k \times n)$; trial division is slow for very large numbers with large prime factors.
- The LCG is predictable and not suitable for security purposes.
- Scratch and Raptor tools from Unit 6 are not part of the program.

13. Future Enhancements

- Add Raptor flowcharts for key algorithms (for example gcd and prime_factors) and a Scratch animation of the sieve.
- Step-by-step trace mode that prints variable values at every iteration.
- Save results and history to a file.
- Support decimal numbers; add binary search and sorting once they are covered in class.

14. References

- Dromey, R. G. *How to Solve it by Computer*. PHI.
- Dierbach, C. *Introduction to Computer Science Using Python: A Computational Problem-Solving Focus*. Wiley, 2015.
- Downey, A. B. *Think Python: How to Think Like a Computer Scientist* (2nd ed.). O'Reilly, 2015.
- Levitin, A. *Introduction to the Design and Analysis of Algorithms* (3rd ed.). Pearson, 2011.
- Kanetkar, Y. *Let Us Python*. BPB Publishers, 2019.
- CSE1021 course syllabus, VIT.
- Python documentation: <https://docs.python.org/3/> (language reference for built-in types and functions).